Roadmap to Become an

Agentic AI Engineer in 2026

Break into Agentic AI with a Step-by-Step Guide

Lamhot Siagian

Complete Roadmap to Become an Agentic AI Engineer in 2026

Interview Questions & Answers by Topic

Lamhot Siagian
PhD Student • AI Evaluation Engineer • AI Engineer
Machine Learning • Data Science & AI/ML
linkedin.com/in/lamhotsiagian
softwaretestarchitect.com
lamhotsiagian2025@gmail.com

January 19, 2026

This document turns the 2026 Agentic AI learning roadmap into practical interview practice. Each section includes 10 common interview questions with model answers and a few small code examples.

Contents

1 How to Use This Roadmap

This roadmap follows a "foundation-first" order: learn core programming, then LLM concepts, then frameworks, then advanced agent architecture, then production deployment.

How to practice: for each topic, (1) read the questions, (2) rewrite answers in your own words, (3) implement at least one small project per section, and (4) keep notes of failures and fixes—that is what interviewers want to hear.

Scope: The questions focus on Agentic AI engineering for software products: tool-using LLM apps, multi-agent workflows, RAG, memory, evaluation, and deployment.

2 Python Fundamentals (for Agentic AI)

1.1 Question: Why is Python the default language for Agentic AI engineering?

Answer: Python has a mature ecosystem for APIs, data processing, and ML (e.g., FastAPI, Pydantic, NumPy, PyTorch) and excellent developer ergonomics. Most agent frameworks and tooling (LangChain/LangGraph, CrewAI, AutoGen, vector DB clients) provide first-class Python support. In interviews, emphasize that Python lets you rapidly prototype and then harden systems with typing, tests, and packaging.

1.2 Question: Explain how you would structure a Python project for an agentic system.

Answer: Use a layered structure: `app/` (API/UI entry points), `core/` (domain logic, prompts, policies), `agents/` (agent graphs, routers), `tools/` (tool wrappers, schemas), `rag/` (chunking, retrieval), `eval/` (tests, golden sets), and `infra/` (Docker, configs). Add `pyproject.toml`, typed interfaces, and unit/integration tests. The goal is separations of concerns so prompts/tools can evolve without breaking deployment.

1.3 Question: What Python features matter most for building robust agents?

Answer: Type hints (mypy/pyright), dataclasses or Pydantic models for schemas, context managers for resource safety, async/await for IO-heavy tool calls, and exceptions for explicit error handling. OOP is useful for tool adapters, but composition and small pure functions often scale better. Also important: logging, retries/backoff, and dependency injection for testability.

1.4 Question: How do you design a clean API client for tools (REST/GraphQL)?

Answer: Define request/response models (Pydantic), centralize auth and base URL, implement timeouts, retries, and idempotency where possible. Expose small methods aligned to business actions, not raw endpoints. Log correlation IDs for tracing across agent steps. In interviews, mention protecting secrets with env vars or a secret manager and never printing tokens.

1.5 Question: When would you use synchronous vs asynchronous Python for agents?

Answer: If tools are mostly network calls (search, DB, external APIs), async can improve throughput and latency by running calls concurrently. If your workload is CPU-bound (embedding large batches locally), multiprocessing or background workers may be better. Many agent apps mix both: async for tool calls, and a job queue for heavy preprocessing/indexing.

1.6 Question: Show a minimal example of a typed tool input schema in Python.

Answer: A good pattern is to validate tool inputs before the agent runs the tool.

```
from pydantic import BaseModel, Field

class WeatherArgs(BaseModel):
    city: str = Field(..., min_length=2)
    units: str = Field("metric", pattern="^(metric|imperial)$")
```

Typed schemas reduce hallucinated parameters and give clear errors you can route back to the agent for self-repair.

1.7 Question: How do you test agentic code where outputs are probabilistic?

Answer: Test deterministic layers (parsers, tool adapters, routing rules) with unit tests. For LLM steps, use "golden" prompts with snapshots, and evaluate with metrics like exact match,

JSON schema validity, or rubric-based scoring. Add integration tests that mock tools and control seeds/temperature. The goal is to detect regressions, not to prove perfect correctness.

1.8 Question: What are common Python pitfalls in production agent apps?

Answer: Unbounded retries causing storms, missing timeouts, leaking file handles/sockets, global state shared across requests, and weak input validation. Another pitfall is mixing prompt logic with business logic so changes become risky. Finally, lacking observability (structured logs, traces) makes debugging "agent went weird" almost impossible.

1.9 Question: How do you manage configuration across local/dev/prod?

Answer: Use a single config object loaded from env vars (and optionally a config file), validated by Pydantic. Keep secrets out of source control. Version configs with infrastructure (Terraform/CloudFormation) and document required variables. In interviews, mention feature flags for safely rolling out new prompts or agent policies.

1.10 Question: Explain dependency management and reproducibility in Python for ML/agents.

Answer: Use a lockfile approach (e.g., uv/poetry/pip-tools) so versions are pinned. Separate runtime deps from dev/test deps. Build Docker images with pinned OS packages. Reproducibility matters because small library changes can alter tokenization, HTTP clients, or vector DB behavior, which changes agent outputs.

3 LLM Fundamentals

2.1 Question: In simple terms, how does an LLM generate text?

Answer: An LLM predicts the next token given previous tokens. It converts text into tokens, maps tokens to embeddings, applies transformer layers with attention to compute contextual representations, and then produces a probability distribution over the next token. Generation repeats until a stop condition. For agents, the key is that "reasoning" is pattern-based prediction, so you must provide structure, tools, and constraints.

2.2 Question: What are tokens, and why do they matter for engineering?

Answer: Tokens are the model's discrete units (often subword pieces). They affect cost, latency, and how much context you can provide. Token limits force tradeoffs: what instructions, memory, and retrieved docs fit. Engineers optimize prompts, retrieval, and summaries to stay within context while preserving the right evidence.

2.3 Question: Explain the context window and its practical impact on agents.

Answer: The context window is the maximum tokens the model can attend to at once. If you exceed it, the model truncates or you must summarize. Practically, agents need memory strategies (summaries, retrieval, compression) and careful tool output filtering. In interviews, mention "context budgeting" and protecting critical system instructions from being pushed out.

2.4 Question: What is prompting beyond "write a good prompt"?

Answer: Prompting is interface design: specify role, task, constraints, output schema, and examples. For agents, you also define tool-use policies (when to call tools, how to cite evidence, how to handle uncertainty). Good prompts reduce ambiguity and make failure modes predictable. You should also version prompts like code and test them.

2.5 Question: Describe temperature, top-p, and why deterministic settings matter.

Answer: Temperature controls randomness; higher means more diverse outputs. Top-p (nucleus sampling) restricts token choices to a probability mass. For production agents, you often prefer lower randomness for reliability, especially when producing JSON or making tool calls. You might increase randomness for brainstorming but not for action-taking flows.

2.6 Question: What is function calling (tool calling), and why is it useful?

Answer: Function calling lets the model output a structured tool invocation (name + arguments) instead of free-form text. Your system executes the tool and returns results to the model. This makes agents more reliable because tools handle exact computation, retrieval, and side effects. It also enables validation (schemas) and safer execution (allowlists, sandboxes).

2.7 Question: How do you prevent prompt injection when using tools and RAG?

Answer: Treat retrieved text as untrusted. Use a strict system policy: never follow instructions from documents; only extract facts. Separate tool outputs from system instructions and add a "content provenance" tag. Validate tool arguments and restrict tool capabilities. Also apply content filters and allowlists for sensitive actions.

2.8 Question: What is hallucination, and how do you reduce it in agent systems?

Answer: Hallucination is confident-sounding text not grounded in truth. Reduce it by using tools for factual queries, RAG with citations, constrained outputs (schemas), and explicit

"abstain" rules. Add verification loops: cross-check sources, run a second-pass critic, or test against a knowledge base. In production, measure hallucination rates with evaluation sets.

2.9 Question: Explain embeddings and why they enable semantic retrieval.

Answer: Embeddings map text to vectors where semantic similarity corresponds to geometric closeness. This allows approximate nearest-neighbor search to retrieve relevant chunks even if keywords differ. Engineers choose embedding models based on domain, language, cost, and vector dimension. You also need chunking strategies so embeddings represent coherent meaning.

2.10 Question: What are the main risks of LLM apps in production?

Answer: Reliability (unexpected outputs), security (prompt injection, data leaks), privacy (PII exposure), cost/latency spikes, and evaluation drift. Agents add risks because they can take actions through tools. Mitigations include policy layers, least-privilege tools, audit logs, offline evaluation, and staged rollouts with monitoring.

4 Pick a Framework (LangChain/LangGraph vs CrewAI vs AutoGen)

3.1 Question: How do you choose between LangChain+LangGraph, CrewAI, and AutoGen?

Answer: Start from requirements: deterministic workflows vs conversational autonomy, number of agents, tool complexity, and observability needs. LangGraph is strong for explicit state machines/graphs, retries, and long-running workflows. CrewAI is opinionated for "role-based" multi-agent collaboration. AutoGen is flexible for agent-to-agent chat patterns. In interviews, say you prototype quickly but stabilize with explicit graphs and tests.

3.2 Question: Why is LangGraph often recommended for production agents?

Answer: It models agent behavior as a graph with nodes (steps) and edges (transitions), which is easier to reason about than implicit loops. You can checkpoint state, enforce policies at boundaries, and add retries. This improves debuggability and prevents runaway conversations. It also supports human-in-the-loop patterns more naturally.

3.3 Question: What is the biggest anti-pattern when adopting a framework?

Answer: Copy-pasting demo code and treating the framework as the architecture. Frameworks are implementation tools; architecture is your state model, tool boundaries, data contracts, and safety rules. If you skip fundamentals (schemas, error handling, evaluation), frameworks will amplify chaos. Interviewers love hearing "I start small and harden layers."

3.4 Question: How do you handle vendor lock-in concerns?

Answer: Abstract the LLM and embedding providers behind interfaces. Avoid embedding provider-specific features unless needed. Keep prompts, schemas, and evaluation sets portable. If using a framework, isolate it in a layer so core business logic doesn't depend on it. Then you can swap frameworks or providers with fewer changes.

3.5 Question: What does "state" mean in an agent graph?

Answer: State is the structured data that flows through steps: user input, conversation history, retrieved documents, tool results, and decisions. Good state design is typed and minimal. It enables reproducibility (replay a run), observability (inspect each field), and safety (validate transitions). Poor state design leads to hidden coupling and brittle behavior.

3.6 Question: Explain how you would implement a router that chooses tools.

Answer: Use a policy: either rules (keywords, intents) or an LLM-based classifier constrained to a small label set. Then validate the chosen tool and arguments against schemas. Log decisions and confidence. A robust pattern is: Router (decide) $\rightarrow$ Tool Executor (act) $\rightarrow$ Verifier (check) before responding.

3.7 Question: How do frameworks help with output structure (JSON, schemas)?

Answer: They provide parsers, output constraints, and utilities to enforce structured outputs. Even without built-in helpers, you can wrap outputs with Pydantic validation. If parsing fails, the framework can route to a repair step. In interviews, mention fail-closed behavior: if schema validation fails, do not execute actions.

3.8 Question: How do you debug agents inside a framework?

Answer: Start with traces: prompts, tool calls, inputs/outputs, latency, and token usage. Reproduce with a fixed seed/temperature. Then isolate failure: was it retrieval, routing, tool

error, or prompt ambiguity? Framework-specific debuggers help, but the core is observability + replay.

3.9 Question: What is a good migration path from a notebook demo to production?

Answer: Extract code into a package, add configuration management, and wrap the agent behind an API. Introduce typed schemas, error handling, retries, and rate limits. Add evaluation harnesses with a small golden dataset. Finally containerize and deploy with monitoring. This staged path prevents "big rewrite" failures.

3.10 Question: What is your default "minimal stack" for agentic prototypes?

Answer: Python + FastAPI, a single agent loop, a small set of tools with strict schemas, a vector store (or even in-memory) for RAG, and basic tracing/logging. When behavior stabilizes, move to an explicit graph (LangGraph), add a UI (Streamlit), and implement evaluations. The key is minimal moving parts at first.

5 Advanced Framework Concepts (LCEL, Runnables, Workflows, Multi-Agent)

4.1 Question: What is LCEL and why do engineers use it?

Answer: LCEL (LangChain Expression Language) composes components (prompts, models, parsers, tools) into pipelines. It encourages modularity: you can swap a model or parser without rewriting everything. It also makes complex chains readable and testable. In interviews, highlight composition and observability benefits.

4.2 Question: What are "runnables" conceptually?

Answer: A runnable is a unit that takes input, produces output, and can be composed with other runnables. Think of it as a functional pipeline building block. This helps you standardize execution, logging, retries, and concurrency. Even outside LangChain, the same idea applies: uniform interfaces for steps.

4.3 Question: How do you design a workflow that includes retries and fallbacks?

Answer: Classify failures (tool timeout vs invalid args vs model parsing error). For transient failures, retry with exponential backoff. For persistent failures, fallback to simpler tools or ask a clarifying question. In graphs, model this explicitly: error edge $\rightarrow$ repair node $\rightarrow$ re-try. Log each attempt to avoid infinite loops.

4.4 Question: Explain "multi-agent" vs "single agent with tools."

Answer: Single agent with tools is one decision-maker calling external functions. Multi-agent splits responsibilities: e.g., planner, retriever, executor, critic. This can improve specialization and safety but increases coordination complexity. Interviewers want to hear that you only go multi-agent when the task truly benefits from decomposition.

4.5 Question: What is a "workflow" compared to a "chain"?

Answer: A chain is usually linear: step A then B then C. A workflow includes branching, loops, human approval steps, and different paths for different conditions. Agentic systems often need workflows because real tasks have uncertainty and partial failures. LangGraph-like state machines are a natural fit.

4.6 Question: How do you prevent agents from looping forever?

Answer: Add maximum steps, time budgets, and "stop conditions" based on task completion signals. Track repeated tool calls or repeated reasoning patterns. Implement a watchdog that forces escalation: ask the user, or return partial results. In a graph, enforce these via state counters and guard edges.

4.7 Question: What is "structured output" and why is it critical for agents?

Answer: Structured output means the model produces machine-validated data (JSON conforming to a schema). It prevents brittle string parsing and reduces hallucinated parameters. It also enables safe tool execution: only run if schema validation passes. For agentic products, structured output is often the difference between a demo and a reliable system.

4.8 Question: How do you design a "critic" or verifier step?

Answer: Define explicit criteria: citation present, tool results used, JSON valid, constraints met. Use deterministic checks first (schema validation, regex, business rules). Optionally add

an LLM judge with a rubric, but keep it as a second layer. If verification fails, route to a repair step or ask for clarification.

4.9 Question: What are the tradeoffs of parallelizing agent steps?

Answer: Parallel tool calls reduce latency but can waste cost if many calls are unnecessary. Parallel LLM calls improve quality via "self-consistency" but increase expense. You should parallelize where uncertainty is high and results are reusable, and serialize where decisions depend on prior results. Always cap concurrency and handle rate limits.

4.10 Question: How do you handle long-running tasks (minutes/hours) with agents?

Answer: Use async jobs with persistent state (DB/queue) and checkpoint after each step. Emit progress events to the UI. Design idempotent tool calls so retries don't duplicate side effects. For workflows, model "resume from checkpoint" so the system can recover after restarts.

6 Memory Management (Short-Term, Long-Term, Checkpointing)

5.1 Question: What is the difference between short-term and long-term memory in agentic AI?

Answer: Short-term memory is the immediate conversation/context window: recent turns, tool outputs, current task state. Long-term memory is stored externally: databases, vector stores, user profiles, summaries. Short-term is fast but limited; long-term is scalable but needs retrieval and relevance filtering. Engineering is choosing what to store and when to retrieve.

5.2 Question: When should you store memory as text summaries vs embeddings?

Answer: Use summaries for "what happened" in a session (decisions, commitments, preferences). Use embeddings for large knowledge where you need semantic retrieval (notes, docs, past tickets). Often you combine both: a summary for quick context plus embeddings for detailed recall. Also consider structured memory (key-value) for stable facts like a user's preferred units or language.

5.3 Question: What is checkpointing and why is it important?

Answer: Checkpointing saves workflow state after steps so you can resume after failures, timeouts, or human approvals. It's critical for long-running agents and for auditability. A good checkpoint includes inputs, tool calls, outputs, and a version of prompts/policies. This enables replay and debugging.

5.4 Question: How do you prevent memory from causing privacy or security issues?

Answer: Apply data minimization: store only what you need. Encrypt at rest, restrict access by tenant, and set retention policies. Avoid storing secrets, credentials, or sensitive PII. If you must store user-specific memory, give users transparency and controls. Also sanitize tool outputs before saving.

5.5 Question: What is "context budgeting" for memory?

Answer: It's deciding how much of the context window to allocate to instructions, recent chat, retrieved docs, and memory. You can enforce budgets: e.g., max 30% for retrieved docs, max 20% for memory summary. When exceeding budgets, compress: summarize, deduplicate, and drop low-value content. A budget prevents critical instructions from being crowded out.

5.6 Question: How do you evaluate whether memory helps or hurts?

Answer: Run A/B tests with and without memory and compare task success, hallucination rate, and user satisfaction. Memory can hurt by introducing outdated or irrelevant facts. Use freshness scoring and conflict resolution rules. In interviews, mention monitoring "memory hit rate" and "memory-induced error" cases.

5.7 Question: Explain "recency" vs "relevance" in memory retrieval.

Answer: Recency prioritizes newer info; relevance prioritizes semantic similarity. In practice you balance both: retrieve top semantic matches, then re-rank by recency and trust. For user preferences, recency can matter (people change their mind). For stable facts, relevance dominates.

5.8 Question: How do you implement memory for multi-agent systems?

Answer: Decide what is shared vs private. Shared memory might include a task plan and verified facts; private memory might include a specialist agent's intermediate notes. Use

structured state passed through the graph as the primary "truth," and store long-term artifacts externally. Always include provenance: where each memory came from and when.

5.9 Question: What are common failure modes of long-term memory?

Answer: Retrieving irrelevant chunks, storing noisy or unverified information, and feedback loops where hallucinations get stored as memory. Also: stale preferences and conflicting memories. Mitigate with validation (store only verified facts), decay/expiration, and a "do not store" policy for uncertain content. A good rule is "only store what you can justify."

5.10 Question: How do you handle user corrections to memory?

Answer: Treat user corrections as high priority. Update structured memory fields and mark old entries as deprecated rather than deleting blindly (for auditability). If using embeddings, store a new corrective note and re-rank by recency. Expose a simple UI/command for users to view and edit what is remembered.

7 Tool Integration (Custom Tools, Connectors, Decorators)

6.1 Question: What makes a tool "agent-friendly"?

Answer: Clear name, narrow purpose, typed input schema, deterministic output, and fast failure. Tools should return structured data, not long narratives. They should enforce timeouts and return helpful error codes. Agent-friendly tools are easy to test and safe to call repeatedly.

6.2 Question: How do you safely expose tools that have side effects (email, purchases, deletes)?

Answer: Use least privilege and separate "read" tools from "write" tools. Require explicit confirmations for irreversible actions. Add policy checks and human-in-the-loop approvals. Log every action with inputs, outputs, and user identity. In interviews, emphasize that the agent should never directly execute high-risk actions without guardrails.

6.3 Question: Explain the role of an allowlist and sandbox for tools.

Answer: An allowlist limits which tools the model can call. A sandbox limits what those tools can do (e.g., restricted filesystem, network egress rules). Together they reduce damage from hallucinated tool calls or prompt injection. It is standard to block arbitrary code execution unless the environment is fully isolated and audited.

6.4 Question: How do you design tool outputs to minimize context bloat?

Answer: Return only what the agent needs: concise fields and summaries. Provide pagination or "top-k" results. Strip HTML, logs, and irrelevant metadata. If needed, store large raw outputs externally and return a short reference ID. This keeps the context window focused and cheaper.

6.5 Question: Show a minimal example of a custom tool wrapper function.

Answer: Keep it deterministic, validated, and timeout-safe.

```
import httpx
from pydantic import BaseModel

class SearchArgs(BaseModel):
    q: str
    k: int = 5

async def web_search(args: SearchArgs) -> dict:
    async with httpx.AsyncClient(timeout=10.0) as client:
        r = await client.get("https://example.com/search", params=args.model_dump())
        r.raise_for_status()
        return r.json()
```

Even if your framework has decorators, the engineering principles are the same.

6.6 Question: How do you handle tool errors so the agent can recover?

Answer: Return structured errors: code, message, and retryable flag. For retryable failures (timeouts), attempt again with backoff. For non-retryable errors (validation), ask the model to repair inputs. Always cap retries and expose the error to logs/traces. An agent that cannot self-repair should degrade gracefully and ask the user.

6.7 Question: What is the difference between "tools" and "plugins/connectors"?

Answer: Tools are callable functions in your runtime. Connectors/plugins often wrap external services with auth and discovery (Google Drive, Slack, Jira). Engineering concerns include OAuth flows, token refresh, and permission scopes. In interviews, stress permission boundaries: the agent can only access what the user authorized.

6.8 Question: How do you version tools and keep backward compatibility?

Answer: Treat tools like APIs. Version schemas (e.g., `tool_v1`, `tool_v2`) or support optional fields. Deprecate gradually and monitor usage. In agents, pin tool versions per workflow so behavior is stable. This prevents silent breakages when tools evolve.

6.9 Question: How do you prevent the model from calling tools unnecessarily?

Answer: Use explicit tool-use policies: "call a tool only when you need external truth." Add a classifier step that chooses "answer directly" vs "use tool." Penalize unnecessary tool calls in evaluation. Also keep tools expensive by default: the agent learns that tools are scarce resources.

6.10 Question: What observability signals are most important for tool integration?

Answer: Tool latency, error rate by tool, retries, request volume, and output sizes. Also track which tool calls correlate with successful task completion. Add trace spans per tool call and include sanitized arguments. This helps you find the bottleneck tool or the tool that causes most agent failures.

8 RAG Systems (Vector Stores, Embeddings, Retrieval Strategies)

7.1 Question: What problem does RAG solve in agentic AI?

Answer: RAG (Retrieval-Augmented Generation) injects external knowledge into the prompt by retrieving relevant documents. It reduces hallucinations and enables up-to-date or private knowledge without retraining. For agent systems, RAG also provides evidence for decisions and tool calls. The best answers cite retrieved sources and avoid inventing missing facts.

7.2 Question: How do you choose chunk size and overlap for indexing documents?

Answer: Chunk size depends on document structure and query patterns. Too small: you lose context; too large: retrieval becomes noisy and expensive. A common starting point is 300–800 tokens with 10–20% overlap, then iterate using evaluation. Use semantic chunking (split by headings) when possible. Always measure retrieval quality, not guess.

7.3 Question: What is the difference between dense, sparse, and hybrid retrieval?

Answer: Dense retrieval uses embeddings to match meaning. Sparse retrieval uses term-based methods (BM25) that excel at exact keyword matches. Hybrid combines both and often improves recall, especially for technical terms. Many production systems retrieve with hybrid then re-rank with a cross-encoder or LLM.

7.4 Question: Explain re-ranking and why it improves RAG.

Answer: Initial retrieval is approximate. Re-ranking uses a stronger model to score candidate chunks against the query. This improves precision of the final context, which improves answer quality and reduces hallucinations. However it costs extra latency. In interviews, mention balancing quality vs latency and caching frequent queries.

7.5 Question: How do you prevent irrelevant retrieval from polluting the answer?

Answer: Use top-k carefully (not too high), apply re-ranking, and filter by metadata (date, source, permissions). Add a "no relevant evidence" condition that triggers abstention or a clarifying question. Also summarize retrieved chunks before final answering. Finally, require citations that map to retrieved chunks to enforce grounding.

7.6 Question: What evaluation metrics matter for RAG?

Answer: Retrieval metrics: recall@k, precision@k, MRR, nDCG. Answer metrics: factuality, citation correctness, task success. Also measure latency, cost, and failure rates. A practical approach is a small curated dataset with expected sources and answers, plus error analysis.

7.7 Question: How do you handle updates to documents and keep the index fresh?

Answer: Use incremental indexing: detect changed documents, re-embed affected chunks, and update metadata. Track document versions and timestamps. For high-change sources, consider streaming ingestion. In interviews, emphasize that stale indexes cause wrong answers, so freshness monitoring is a production requirement.

7.8 Question: What is metadata filtering and why is it critical?

Answer: Metadata filtering restricts retrieval by fields like tenant, permission, doc type, date, or language. It prevents data leaks across users and improves relevance. You should enforce metadata filters outside the model (in code) so they cannot be bypassed. This is a key security requirement for enterprise RAG.

7.9 Question: Explain "grounded generation" and citations in RAG.

Answer: Grounded generation means the model's claims should be supported by retrieved evidence. Citations map statements to sources (chunk IDs or URLs). You can enforce citations by formatting retrieved chunks with IDs and requiring the model to reference them. Then you can automatically verify citations to reduce hallucinated references.

7.10 Question: What are common RAG failure cases and how do you fix them?

Answer: Bad chunking, weak embeddings for the domain/language, missing metadata filters, and top-k too high/low. Also query mismatch: user asks for "how" but retrieval finds "what." Fixes include better chunking, hybrid retrieval, re-ranking, prompt changes, and adding query rewriting. Always do error analysis with real queries and logs.

9 Agents & Multi-Agents (ReAct, Supervisors, Communication)

8.1 Question: What is the ReAct pattern?

Answer: ReAct interleaves reasoning (plan) with actions (tool calls) and observations (tool results). The agent iteratively decides what to do next based on observations. It improves factuality because the agent can look things up instead of guessing. Engineering challenge: controlling loops, tool misuse, and context growth.

8.2 Question: What is a supervisor agent and when do you use one?

Answer: A supervisor coordinates specialist agents (retriever, coder, critic) by routing tasks and merging outputs. Use it when tasks are complex enough to benefit from specialization. However, supervisors add overhead and can hide errors if not instrumented. A good design is a supervisor with explicit criteria for delegating and accepting results.

8.3 Question: How do agents communicate safely and effectively?

Answer: Use structured messages: objective, constraints, current state, and required output format. Avoid passing huge raw context; pass references and summaries. Enforce role boundaries so agents don't override system policies. Also track provenance: which agent produced which claim. This supports debugging and accountability.

8.4 Question: What is the difference between "planning" and "execution" in agents?

Answer: Planning chooses steps and tools; execution performs them. Separating them reduces risk: a planner can propose actions, but an executor validates and runs only safe actions. This is similar to "dry run" then "commit." Many production systems keep planning low-temperature and execution strictly schema-validated.

8.5 Question: How do you handle uncertainty in agent decisions?

Answer: Make uncertainty explicit: confidence scores, assumptions, and "need more info" flags. Add policies: if confidence is low, call a tool or ask a clarifying question. Avoid forcing a single guess. In interviews, mention that uncertainty handling is a reliability feature, not a weakness.

8.6 Question: How do you prevent tool abuse in multi-agent setups?

Answer: Centralize tool execution behind a policy gate. Even if a specialist agent requests a tool, the executor enforces allowlists, scopes, and rate limits. Log all requests and rejections. Also restrict each agent's tool set to what it needs. This reduces blast radius if one agent is compromised or confused.

8.7 Question: What is an "agent protocol" and why does it matter?

Answer: It's the standardized format agents use for inputs/outputs (schemas, fields, error conventions). Protocols reduce miscommunication and make system behavior predictable. They also allow swapping agents or models without breaking workflows. A simple protocol includes: goal, constraints, context refs, tool results, and final answer with citations.

8.8 Question: Describe a practical multi-agent design for RAG + writing a final report.

Answer: Use a retriever agent to gather and rank sources, a summarizer agent to extract key points with citations, a writer agent to draft the report, and a critic agent to check for unsupported claims. The supervisor orchestrates: retrieve $\rightarrow$ summarize $\rightarrow$ draft $\rightarrow$ verify. If verification fails, loop back to retrieval. This pattern balances quality and grounding.

8.9 Question: How do you evaluate an agent beyond "it seems good"?

Answer: Create task suites: realistic user goals with expected outcomes. Measure success rate, tool call counts, latency/cost, and safety violations. Add qualitative error buckets (retrieval failure, wrong tool, bad reasoning, unsafe action). Use regression tests on prompts and policies. Evaluation is continuous because tools and models change.

8.10 Question: What are the key safety concerns unique to agents?

Answer: Agents can take actions (write files, send messages, make changes), so errors have real consequences. Prompt injection can redirect actions. Tool outputs can contain malicious instructions. Mitigations: least privilege, confirmations, sandboxing, policy gates, and audit logs. Safety is an engineering layer, not a prompt-only problem.

10 Build Real-World Projects (FastAPI, Streamlit/UI, Docker, AWS)

9.1 Question: Describe an end-to-end architecture for a production agent app.

Answer: A typical architecture: UI (Streamlit/React) → API (FastAPI) → Agent Orchestrator (graph) → Tools (internal services/APIs) + RAG (vector store) + Memory (DB). Add observability (logs, traces, metrics) and evaluation pipelines. Use a queue for long tasks and a cache for repeated retrieval. Security includes auth, tenant isolation, and secret management.

9.2 Question: Why FastAPI is a common choice for agent backends?

Answer: FastAPI is fast to develop, supports async, has strong typing via Pydantic, and generates OpenAPI docs. This makes tool endpoints and agent APIs well-defined. It also integrates well with dependency injection and middleware for logging/auth. In interviews, mention input validation and schema-driven contracts.

9.3 Question: How do you build a simple Streamlit UI for an agent?

Answer: Start with a chat layout: input box, message history, and a "debug" panel for traces. Send user messages to FastAPI, stream responses, and display citations/tool steps. Keep user state minimal and rely on backend session IDs. A good UI makes failures visible so users can correct quickly.

9.4 Question: What should be in a Dockerfile for an agentic app?

Answer: Use a slim base image, pin Python dependencies, and copy only necessary files. Set env vars for configuration and run as a non-root user. Add health checks and expose ports. Build separate images for API and worker if you have background jobs. In interviews, mention reproducible builds and vulnerability scanning.

9.5 Question: How would you deploy this on AWS?

Answer: Common options: ECS/Fargate for containers, EKS for Kubernetes, or Lambda for small serverless APIs. Use RDS/DynamoDB for state, S3 for artifacts, and a managed vector store or self-hosted one. Use Secrets Manager for credentials. Add CloudWatch logs/metrics and alarms. Pick the simplest service that meets scaling and compliance needs.

9.6 Question: What observability do you add for agent debugging?

Answer: Structured logs with request IDs, tracing spans for each agent step and tool call, and metrics for latency, token usage, and errors. Capture prompts and tool arguments in a sanitized form. Add a replay mechanism: given a trace ID, reproduce the run. This is essential for diagnosing intermittent failures.

9.7 Question: How do you handle streaming responses to users?

Answer: Stream tokens from the model when possible for better UX. Still, keep tool calls non-streaming or stream progress events ("Searching...", "Calling API..."). Ensure backpressure and timeouts. If streaming fails, fallback to a full response. In interviews, mention that streaming does not replace correctness or citations.

9.8 Question: How do you secure an agent API exposed to the internet?

Answer: Require authentication, rate limit per user, and enforce tenant isolation. Validate all inputs. Restrict tool scopes based on user permissions. Sanitize logs to avoid leaking secrets.

Also defend against prompt injection by treating user content as untrusted and enforcing system policies.

9.9 Question: How do you set up CI/CD for an agent project?

Answer: Run linting, type checks, unit tests, and integration tests (mocked tools). Build and scan Docker images. Deploy to a staging environment with canary prompts/policies. Run evaluation suites on each change and block deploys if metrics regress. This treats prompts and policies like code.

9.10 Question: What does "production readiness" mean for agentic AI?

Answer: It means reliability, safety, observability, and maintainability. The agent should degrade gracefully, cite evidence, and avoid unsafe actions. You should be able to reproduce failures from logs. Costs and latency must be controlled. Most importantly, you continuously evaluate the system as models, tools, and data evolve.

11 Quick Checklist: The Right Order to Learn (2026)

1. Python fundamentals: types, APIs, async, project structure, tests.
2. LLM fundamentals: tokens, context budgeting, prompting, tool calling.
3. Framework choice: start simple, then graduate to graphs/workflows.
4. Advanced concepts: composition, retries, fallbacks, verification.
5. Memory: summaries + vector retrieval + checkpointing.
6. Tools: schemas, safety gates, observability.
7. RAG: chunking, hybrid retrieval, re-ranking, evaluation.
8. Agents: ReAct, supervisors, protocols, safety.
9. Real projects: FastAPI + UI + Docker + cloud + CI/CD + eval.

Tip for interviews: Bring 2–3 concrete projects (even small) that show tool use, RAG, evaluation, and production thinking. Be ready to explain one failure you debugged (retrieval noise, tool timeout, schema parsing) and how you fixed it.